

SQLite SQL 실습

대학 수강관리 데이터로 배우는
조회와 데이터 변경

03

**DATABASE SYSTEM
CONCEPTS 7E**

SQL 개요

SQL의 정의

Structured Query Language

관계형 데이터베이스의 구조를 정의하고
데이터를 조회·추가·수정·삭제하는 언어입니다.

테이블과 열, 조건을 SQL 문장으로 표현합니다.
DBMS가 문장을 해석하고 실행합니다.

SQL은 데이터베이스를 다루기 위한 표준화된 언어입니다.

SQL의 선언적 질의 방식

컴퓨터공학과(학과 1) 학생의 이름을 조회한다.

```
SELECT name  
FROM student  
WHERE department_id = 1;
```

사용자는 필요한 열과 조건을 지정합니다.
DBMS는 데이터를 찾는 실행 방법을 결정합니다.

SQL에서는 원하는 결과를 기술합니다.

SQL의 역사

1970 관계형 모델

Edgar F. Codd가 관계형 데이터 모델을 제안했습니다.

1974 SEQUEL

IBM의 Donald Chamberlin과 Raymond Boyce가 발표했습니다.

System R과 SQL

IBM의 관계형 DBMS 연구에서 발전했고, 이름을 SQL로 바꿨습니다.

1986·1987 표준화

1986년 ANSI 표준, 1987년 ISO 표준으로 채택되었습니다.

자료: IBM Research의 SEQUEL 논문(1974), IBM SQL 역사, ISO 9075:1987

SQL · SQLite · DBeaver의 역할

SQL 언어

테이블 정의와 데이터 조회 · 변경 명령을 작성합니다.

SQLite DBMS

SQL을 실행하고 university.db의 데이터를 관리합니다.

DBeaver 데이터베이스 도구

DB에 연결해 SQL을 작성하고 결과를 확인하는 화면을 제공합니다.

오늘은 DBeaver에서 SQL을 작성하고, SQLite가 그 문장을 실행합니다.

오늘의 실습 흐름

SQL 개요와 실습 준비 15분

SQL의 정의·역사, DB 연결과 외래 키 설정

조건 조회 30분

01_select.sql의 Q01~Q18 실행과 결과 비교

데이터 변경과 오류 확인 30분

02 실습 후 03 오류를 한 문장씩 실행

개별 문제와 제출 15분

04_exercises.sql에 답안을 작성

실습 준비와 DBeaver 연결

실습 파일과 시작 위치

저장소의 Codes/week03-01 폴더

university.db 준비된 샘플 데이터베이스

01_select.sql 조회 예제 18개

02_create_insert_update_delete.sql 생성·변경

03_errors.sql 의도적인 오류 8개

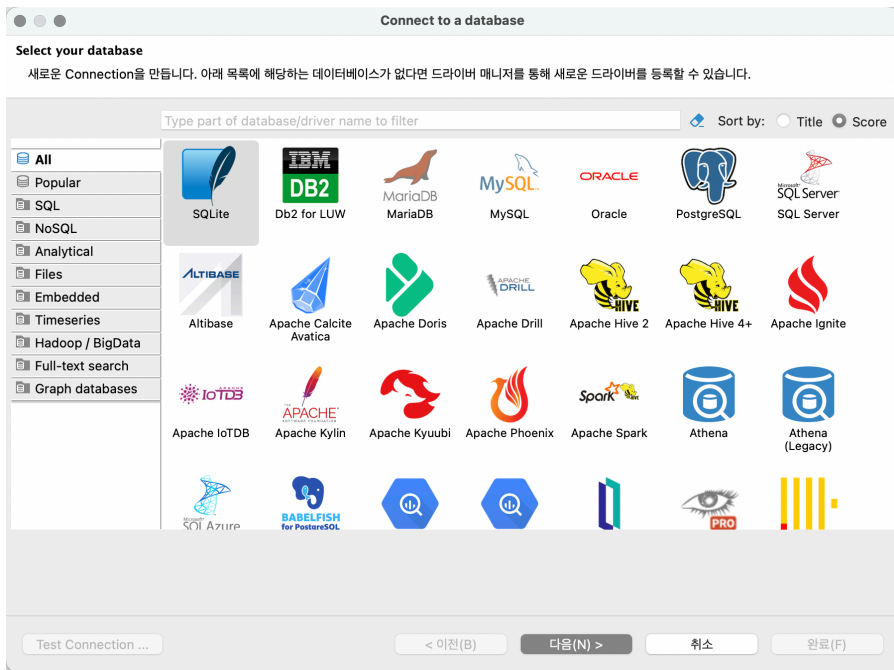
04_exercises.sql 개별 문제

05_expected_results.md 조회 예상 결과

GitHub의 DB 파일은 Download raw로 받습니다. ZIP은 압축 해제 후 사용합니다.

DBeaver 1 SQLite 연결 선택

새 데이터베이스 연결 창에서 SQLite를 선택하고 다음을 누릅니다.



연결 대상
SQLite

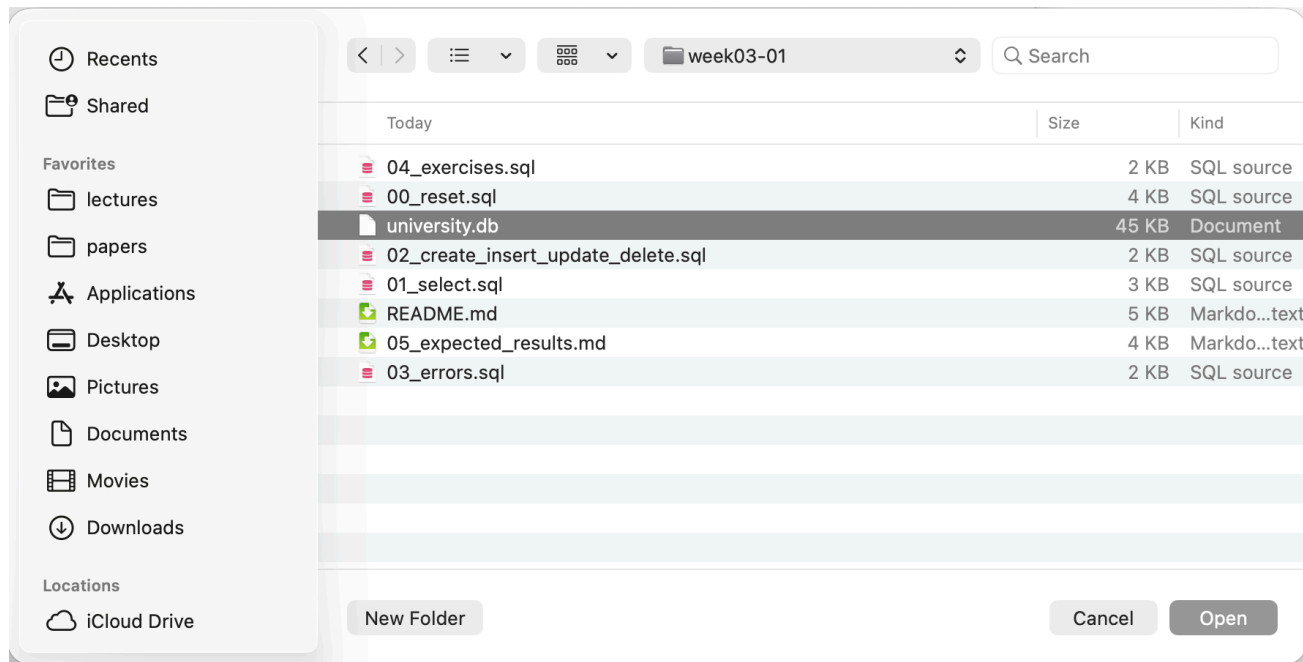
이번 실습은
파일 기반 DB입니다.

제공된 DB 파일로
연결합니다.

화면은 macOS용 DBeaver 캡처입니다. 운영체제에 따라 버튼 위치가 다를 수 있습니다.

DBeaver 2 DB 파일 선택

연결 설정의 파일 선택 창에서 university.db를 선택합니다.



Codes/week03-01/university.db를 선택하고 Open을 누른 뒤 연결 설정을 완료합니다.

DBeaver 3 연결과 학생 데이터

왼쪽 university.db를 펼친 뒤 student 테이블의 Data 탭을 엽니다.

The screenshot shows the DBeaver interface with the 'student' table selected in the 'Data' tab. The table data is as follows:

	123 student_id	A-Z name	A-Z email	123 department_id	123 entry_year	123 active
1	1,001	Kim Minseo	minseo@example.edu	1	2,024	1
2	1,002	Lee Jihoon	jihoon@example.edu	1	2,023	1
3	1,003	Park Soyeon	soyeon@example.edu	2	2,024	1
4	1,004	Choi Junho	junho@example.edu	3	2,022	1
5	1,005	Jung Haeun	haeun@example.edu	1	2,022	1
6	1,006	Han Yujin	yujin@example.edu	4	2,023	1
7	1,007	Yoon Seojun	seojun@example.edu	2	2,021	0
8	1,008	Lim Chaewon	chaewon@example.edu	[NULL]	2,025	1

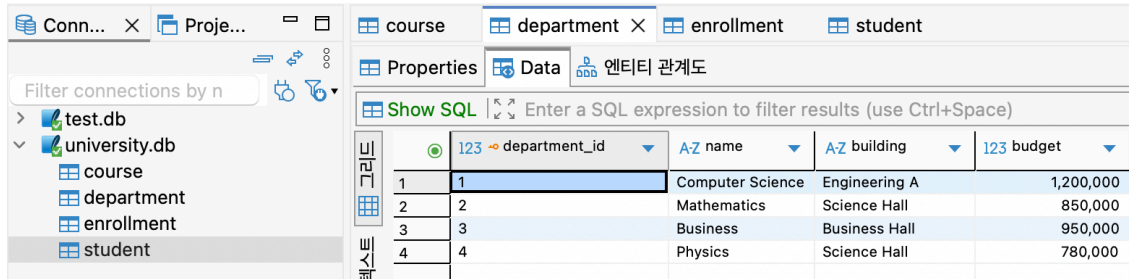
확인할 내용

학생 8행 / 1008 학생의 department_id는 NULL

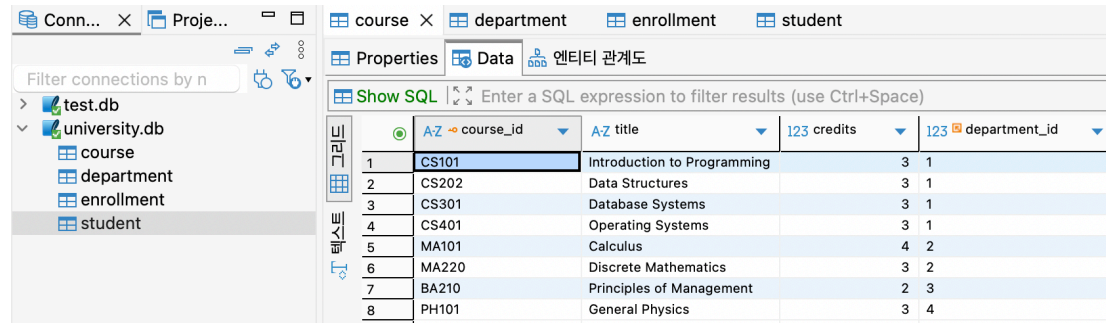
연결 이름이 university.db인지 확인합니다. 이전 수업의 test.db와 구분합니다.

DBeaver 4 학과와 과목 데이터

department와 course를 각각 열어 행 수와 열 이름을 확인합니다.



	123 department_id	A-Z name	A-Z building	123 budget
1	1	Computer Science	Engineering A	1,200,000
2	2	Mathematics	Science Hall	850,000
3	3	Business	Business Hall	950,000
4	4	Physics	Science Hall	780,000

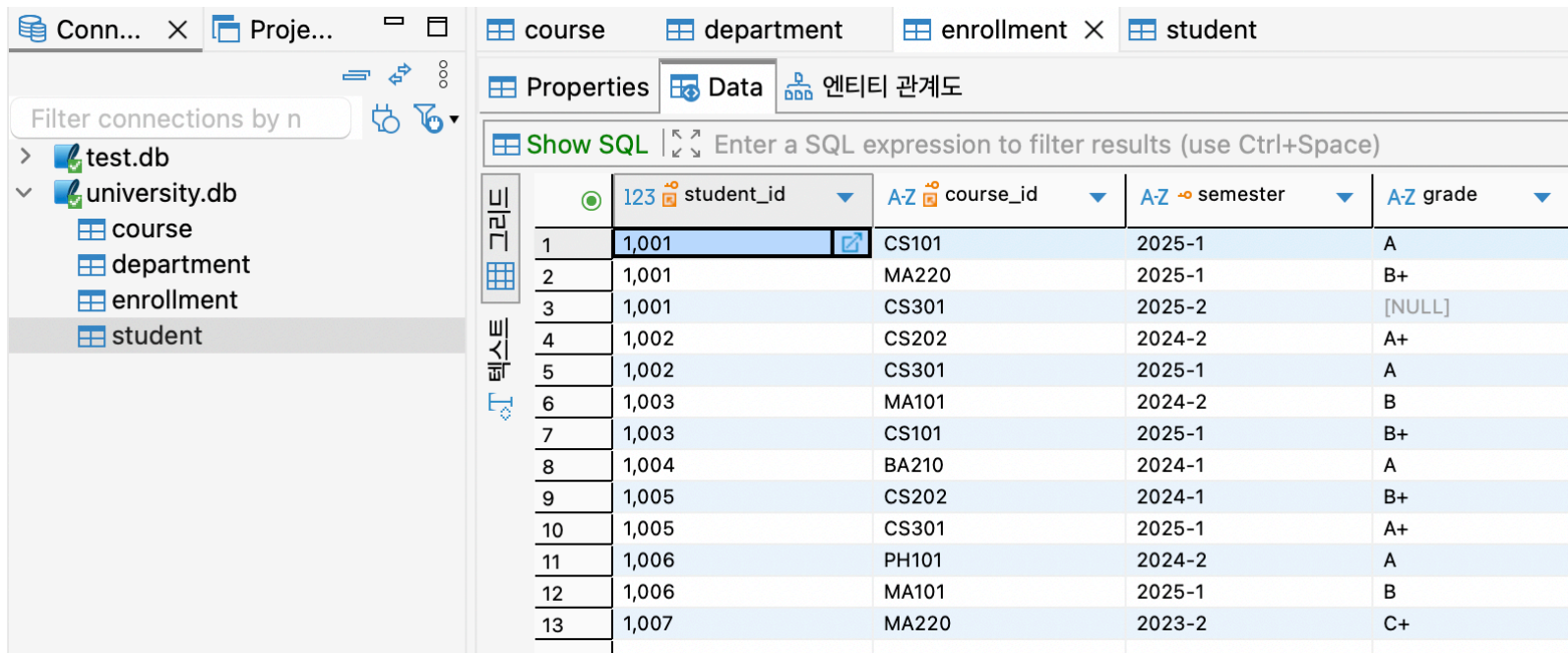


	A-Z course_id	A-Z title	123 credits	123 department_id
1	CS101	Introduction to Programming	3	1
2	CS202	Data Structures	3	1
3	CS301	Database Systems	3	1
4	CS401	Operating Systems	3	1
5	MA101	Calculus	4	2
6	MA220	Discrete Mathematics	3	2
7	BA210	Principles of Management	2	3
8	PH101	General Physics	3	4

department는 4행, course는 8행입니다. 학과 번호는 student·course의 외래 키와 연결됩니다.

DBeaver 5 수강 기록 확인

enrollment에서 학번, 과목 코드, 학기와 성적을 확인합니다.



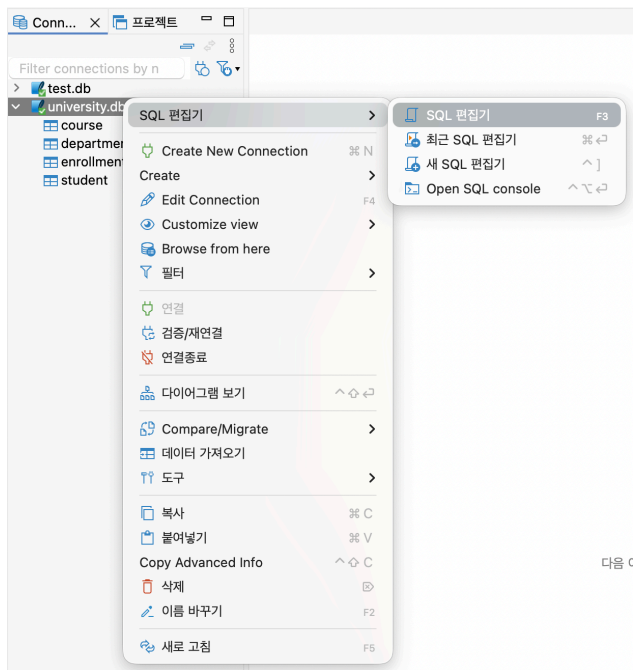
The screenshot shows the DBeaver 5 interface with the 'enrollment' table selected in the 'Data' tab. The table contains 13 rows of data. The first row (row 1) is highlighted, showing student_id 1,001, course_id CS101, semester 2025-1, and grade A. The second row (row 2) shows student_id 1,001, course_id MA220, semester 2025-1, and grade B+. The third row (row 3) shows student_id 1,001, course_id CS301, semester 2025-2, and grade [NULL]. The fourth row (row 4) shows student_id 1,002, course_id CS202, semester 2024-2, and grade A+. The fifth row (row 5) shows student_id 1,002, course_id CS301, semester 2025-1, and grade A. The sixth row (row 6) shows student_id 1,003, course_id MA101, semester 2024-2, and grade B. The seventh row (row 7) shows student_id 1,003, course_id CS101, semester 2025-1, and grade B+. The eighth row (row 8) shows student_id 1,004, course_id BA210, semester 2024-1, and grade A. The ninth row (row 9) shows student_id 1,005, course_id CS202, semester 2024-1, and grade B+. The tenth row (row 10) shows student_id 1,005, course_id CS301, semester 2025-1, and grade A+. The eleventh row (row 11) shows student_id 1,006, course_id PH101, semester 2024-2, and grade A. The twelfth row (row 12) shows student_id 1,006, course_id MA101, semester 2025-1, and grade B. The thirteenth row (row 13) shows student_id 1,007, course_id MA220, semester 2023-2, and grade C+.

	student_id	course_id	semester	grade
1	1,001	CS101	2025-1	A
2	1,001	MA220	2025-1	B+
3	1,001	CS301	2025-2	[NULL]
4	1,002	CS202	2024-2	A+
5	1,002	CS301	2025-1	A
6	1,003	MA101	2024-2	B
7	1,003	CS101	2025-1	B+
8	1,004	BA210	2024-1	A
9	1,005	CS202	2024-1	B+
10	1,005	CS301	2025-1	A+
11	1,006	PH101	2024-2	A
12	1,006	MA101	2025-1	B
13	1,007	MA220	2023-2	C+

수강 기록 13행. 1001 학생의 CS301, 2025-2 성적은 아직 미확정이므로 NULL입니다.

DBeaver 6 SQL 편집기 열기

university.db를 우클릭하고 SQL 편집기 메뉴를 엽니다.



새 SQL 편집기

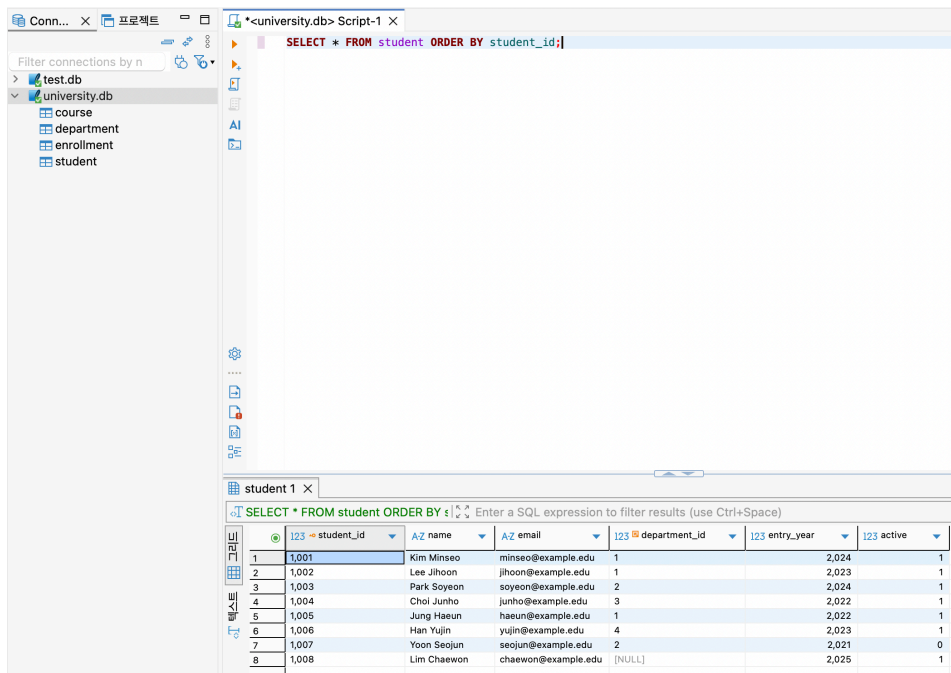
메뉴에서 새 SQL 편집기를 선택합니다.

편집기의 연결 대상이 university.db인지 확인합니다.

SQL 편집기에서 문장을 작성하고 실행합니다. 데이터 필터 입력란과 구분합니다.

DBeaver 7 첫 SELECT 실행

SQL을 입력한 뒤 해당 문장에 커서를 두고 SQL 실행 버튼을 누릅니다.



Q01 실행문

```
SELECT *
FROM student
ORDER BY
    student_id;
```

하단 결과 창에서
학생 8행과
학번 순서를 확인합니다.

다음 슬라이드의 버전·외래 키 설정을 확인한 뒤 Q01~Q18 실습을 진행합니다.

SQLite 연결과 설정

university.db를 열고 아래 문장을 하나씩 실행합니다.

```
SELECT sqlite_version();
```

```
PRAGMA foreign_keys = ON;  
PRAGMA foreign_keys;
```

버전 3.37.0 이상, foreign_keys 결과 1을 확인합니다.

새 연결마다 외래 키를 켭니다. 진행 중인 트랜잭션 밖에서 설정합니다.

샘플 데이터의 구조

department 4행

학과 번호, 학과명, 건물, 예산

student 8행

학번, 이름, 이메일, 소속 학과, 입학 연도, 재학 여부

course 8행

과목 코드, 과목명, 학점, 개설 학과

enrollment 13행

학번, 과목 코드, 학기, 성적

학과 1=컴퓨터공학, 2=수학, 3=경영, 4=물리. 모든 데이터는 가상입니다.

실행 전 결과 예측

각 쿼리에서 확인할 내용

어느 테이블에서 읽는가?

어떤 열을 출력하는가?

어떤 행이 조건을 만족하는가?

결과는 몇 행이고 어떤 순서인가?

예측을 적고 실행한 뒤 05_expected_results.md와 비교합니다.

SELECT와 조건 조회

Q01 전체 학생 조회

학번 순서로 8행

```
SELECT *  
FROM student  
ORDER BY student_id;
```

결과 8행 / student_id: 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008

*는 모든 열입니다. student에는 8명의 학생이 있습니다.

Q02 필요한 열과 별칭

학번·이름만 조회

```
SELECT student_id AS 학번, name AS 이름  
FROM student  
ORDER BY student_id;
```

결과 8행 / 학번: 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008

AS는 결과 열 이름을 바꿉니다. 원본 열 이름은 그대로입니다.

Q03 WHERE

컴퓨터공학과(1) 학생

```
SELECT student_id, name  
FROM student  
WHERE department_id = 1  
ORDER BY student_id;
```

결과 3행 / student_id: 1001, 1002, 1005

학과 번호 1에 해당하는 학생만 남습니다.

Q04 비교

2024년 이후 입학생

```
SELECT student_id, name, entry_year  
FROM student  
WHERE entry_year >= 2024  
ORDER BY student_id;
```

결과 3행 / student_id: 1001, 1003, 1008

>=는 기준값 2024도 포함합니다.

Q05 AND

컴퓨터공학과이면서 2023년 이후 입학

```
SELECT student_id, name  
FROM student  
WHERE department_id = 1 AND entry_year >= 2023  
ORDER BY student_id;
```

결과 2행 / student_id: 1001, 1002

AND는 두 조건을 모두 만족해야 합니다.

Q06 괄호·OR

학과 1 또는 2이며 2024년 입학

```
SELECT student_id, name  
FROM student  
WHERE (department_id = 1  
       OR department_id = 2)  
       AND entry_year = 2024  
ORDER BY student_id;
```

결과 2행 / student_id: 1001, 1003

OR 조건을 괄호로 묶은 뒤 입학 연도를 검사합니다.

Q07 IN

수학과(2) 또는 물리학과(4)

```
SELECT student_id, name  
FROM student  
WHERE department_id IN (2, 4)  
ORDER BY student_id;
```

결과 3행 / student_id: 1003, 1006, 1007

IN은 목록 중 하나와 같은지를 검사합니다.

Q08 BETWEEN

2022~2023년 입학 (양 끝 포함)

```
SELECT student_id, name, entry_year  
FROM student  
WHERE entry_year BETWEEN 2022 AND 2023  
ORDER BY student_id;
```

결과 4행 / student_id: 1002, 1004, 1005, 1006

BETWEEN은 2022와 2023을 모두 포함합니다.

Q09 LIKE

Kim으로 시작하는 이름 (%는 0개 이상의 문자)

```
SELECT student_id, name  
FROM student  
WHERE name LIKE 'Kim%'  
ORDER BY student_id;
```

결과 1행 / student_id: 1001

%는 길이 0 이상의 문자열과 일치합니다.

Q10 LIKE

CS 다음 정확히 세 글자인 과목 코드 (_는 문자 한 개)

```
SELECT course_id, title  
FROM course  
WHERE course_id LIKE 'CS____'  
ORDER BY course_id;
```

결과 4행 / course_id: CS101, CS202, CS301, CS401

_는 정확히 한 문자입니다. CS 뒤 세 문자를 검사합니다.

Q11 DISTINCT

입학 연도 중복 제거

```
SELECT DISTINCT entry_year  
FROM student  
ORDER BY entry_year;
```

결과 5행 / entry_year: 2021, 2022, 2023, 2024, 2025

DISTINCT는 선택한 결과 열 조합의 중복을 제거합니다.

Q12 ORDER BY

최신 입학 연도 우선, 같은 연도는 학번 오름차순

```
SELECT student_id, name, entry_year  
FROM student  
ORDER BY entry_year DESC, student_id ASC;
```

결과 8행 / student_id: 1008, 1001, 1003, 1002, 1006, 1004, 1005, 1007

동률일 때의 학번 순서까지 지정하면 결과를 비교하기 쉽습니다.

Q13 LIMIT

위 정렬의 처음 3명

```
SELECT student_id, name, entry_year  
FROM student  
ORDER BY entry_year DESC, student_id ASC  
LIMIT 3;
```

결과 3행 / student_id: 1008, 1001, 1003

먼저 정렬한 결과에서 처음 3행을 가져옵니다.

Q14 계산식

과목별 주당 수업시간(1학점=50분 가정)

```
SELECT course_id, credits, credits * 50 AS weekly_minutes  
FROM course  
ORDER BY course_id;
```

결과 8행 / course_id: BA210, CS101, CS202, CS301, CS401, MA101, MA220, PH101

계산한 값을 출력하며, 저장된 credits 값은 바뀌지 않습니다.

Q15 NULL 맛보기

소속 학과가 없는 학생

```
SELECT student_id, name  
FROM student  
WHERE department_id IS NULL  
ORDER BY student_id;
```

결과 1행 / student_id: 1008

학과 미정인 학생의 department_id는 NULL입니다.

Q16 NULL 비교 실수

오류는 없지만 0행. Q15와 비교

```
SELECT student_id, name  
FROM student  
WHERE department_id = NULL  
ORDER BY student_id;
```

결과 0행

NULL과 =로 비교하면 TRUE가 되지 않습니다. IS NULL로 고칩니다.

Q17 부정 조건

현재 재학하지 않는 학생

```
SELECT student_id, name  
FROM student  
WHERE active <> 1  
ORDER BY student_id;
```

결과 1행 / student_id: 1007

active는 1=재학, 0=비재학입니다.

Q18 조건과 정렬 조합

3학점 이상 과목, 학점 내림차순·코드 오름차순

```
SELECT course_id, title, credits  
FROM course  
WHERE credits >= 3  
ORDER BY credits DESC, course_id;
```

결과 7행 / course_id: MA101, CS101, CS202, CS301, CS401, MA220, PH101

조건으로 행을 고른 뒤 학점과 코드 순서로 정렬합니다.

테이블 생성과 데이터 변경

DDL과 DML

CREATE TABLE

테이블의 열, 자료형, 제약조건을 정의합니다.

INSERT

새 행을 추가합니다.

UPDATE

조건에 맞는 행의 값을 변경합니다.

DELETE

조건에 맞는 행을 삭제합니다.

02 파일은 `practice_student`만 변경합니다. 원본 `student`와 구분합니다.

STEP 1 연습용 테이블 생성

```
DROP TABLE IF EXISTS practice_student;
CREATE TABLE practice_student (
  student_id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  department_id INTEGER
    REFERENCES department(department_id),
  entry_year INTEGER NOT NULL
    CHECK (entry_year BETWEEN 2000 AND 2100),
  active INTEGER NOT NULL DEFAULT 1
    CHECK (active IN (0, 1))
) STRICT;
```

02 파일 STEP 1과 같은 정의입니다. 다시 실행하면 연습 데이터가 삭제됩니다.

STEP 2 INSERT와 기본값

```
INSERT INTO practice_student
(student_id, name, email,
 department_id, entry_year)
VALUES
(9001, 'Kim Practice', 'practice1@example.edu', 1, 2026),
(9002, 'Lee Practice', 'practice2@example.edu', 2, 2025),
(9003, 'Park Practice', 'practice3@example.edu', NULL, 2026);
```

```
SELECT * FROM practice_student
ORDER BY student_id;
```

결과 3행. active는 생략했으므로 모두 1입니다. NULL은 따옴표 없이 씁니다.

STEP 3 UPDATE 대상 확인

```
SELECT * FROM practice_student  
WHERE student_id = 9002;
```

```
UPDATE practice_student  
SET department_id = 1, active = 0  
WHERE student_id = 9002;
```

```
SELECT * FROM practice_student  
WHERE student_id = 9002;
```

9002 한 명만 학과 1, active 0으로 바꿉니다. 같은 WHERE로 전후를 확인합니다.

STEP 4 DELETE 대상 확인

```
SELECT * FROM practice_student  
WHERE student_id = 9003;
```

```
DELETE FROM practice_student  
WHERE student_id = 9003;
```

```
SELECT * FROM practice_student  
ORDER BY student_id;
```

결과는 9001과 9002 두 행입니다. DELETE 후에도 테이블 정의는 남습니다.

STEP 5 WHERE 없는 UPDATE

```
BEGIN;  
UPDATE practice_student SET active = 0;  
SELECT * FROM practice_student  
ORDER BY student_id;
```

```
ROLLBACK;  
SELECT * FROM practice_student  
ORDER BY student_id;
```

변경 직후: 두 학생 모두 0. ROLLBACK 후: 9001은 1, 9002는 0.

SQL 오류 확인

오류 재현 실습

03_errors.sql의 E01~E08

먼저 PRAGMA foreign_keys 결과 1 확인
오류 블록 하나만 선택하여 실행
오류 메시지에서 제약조건 또는 위치 확인
원인을 한 문장으로 적고 수정 방법 설명

이 파일은 의도적으로 실패합니다. 전체 실행 대신 오류 하나씩 확인합니다.

E01~E04 키와 필수 값

E01 PRIMARY KEY

이미 존재하는 학번 1001을 다시 넣으면 실패합니다.

E02 NOT NULL

name에 NULL을 넣으면 실패합니다.

E03 UNIQUE

이미 존재하는 이메일을 넣으면 실패합니다.

E04 FOREIGN KEY

존재하지 않는 학과 번호 99를 참조하면 실패합니다.

E05~E08 값과 SQL 표현

E05 CHECK

active에 2를 넣으면 허용 범위 0, 1을 벗어납니다.

E06 STRICT 자료형

정수 entry_year에 'hello'를 넣으면 실패합니다.

E07 열 이름

student_name은 이 스키마에 없는 열입니다.

E08 문법

SELEC은 SELECT의 오타입니다.

개별 실습

P01~P04 직접 작성

P01 재학 중인 학생

학번과 이름을 학번 오름차순으로 조회

P02 컴퓨터공학과와 이전 입학생

학과 1이면서 2023년 이전 입학한 학생의 학번·이름 조회

P03 과목명 검색

제목에 Management가 포함된 과목의 코드·제목 조회

P04 학점 종류

과목의 학점을 중복 없이 오름차순으로 조회

04_exercises.sql에 직접 SQL을 작성합니다.

P05~P08 조건 조합

P05 가장 오래된 입학생 2명

학번·이름·입학 연도 조회, 입학 연도와 학번 오름차순

P06 학과가 정해진 최근 입학생

학과가 있고 2024년 이후 입학한 학생의 학번·이름 조회

P07 AND와 OR 수정

학과 1 또는 2이면서 2024년 입학한 학생을 찾도록 수정

P08 3학점이 아닌 과목

3학점 미만 또는 초과인 과목의 코드·학점을 코드 순 조회

조건뿐 아니라 출력 열과 정렬 기준도 문제에 맞춥니다.

P09~P12 변경과 오류 설명

P09 새 학생 추가

9201, 'New Student', 'new@example.edu', 학과 3, 2026년

P10 소속 변경

9201의 학과를 2로 변경하고 해당 학생 조회

P11 학생 삭제

9201을 삭제하고 조회 결과 0행 확인

P12 오류 수정

03 파일에서 오류 하나를 골라 원인과 수정문 작성

02 실습 후 practice_student에서 진행합니다. P09의 active는 기본값을 사용합니다.

초기화와 제출

초기화와 다시 실습하기

원본 데이터가 달라졌을 때

진행 중인 트랜잭션을 종료합니다.

수업용 DB에서 00_reset.sql 전체를 실행합니다.

원본 4개 테이블과 데이터가 다시 만들어집니다.

02 파일을 실행하면 연습용 테이블도 다시 생깁니다.

초기화는 실습 변경 내용을 삭제합니다. 작성한 답안 SQL은 별도 파일에 저장합니다.

실습 제출

학번_week03.sql

P01~P11: 번호별 SQL 답안

P12: 오류 원인과 수정문을 주석으로 작성
조회 결과가 다르면 조건과 정렬을 다시 확인

다음 실습: 같은 데이터로 JOIN과 GROUP BY를 실행합니다.